

ADVANCEMENTS IN MUSIC MOOD CLASSIFICATION MACHINE LEARNING TECHNIQUES FOR MOOD ANALYSIS

*A Major Project Report submitted to
JNTU Hyderabad in partial fulfilment
of the requirements for the award of the degree*

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

JUNNU VIJAYA SRI	21S11A05C6
JUMBARATH KEERTHANA	21S11A0583
VASAM VARSHINI	21S11A05C3
KAMSALI SANDEEP	21S11A05A3

Under the Guidance of

Mr. ANAND

B. Tech, M. Tech.

ASST.PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE

(Approved by AICTE New Delhi and Affiliated to JNTUH)

(Accredited by NBA & NAAC with “A” Grade)

An ISO 9001: 2015 Certified Institution

Maisammaguda, Medchal (M), Hyderabad-500100, T. S.

APRIL 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE

(Approved by AICTE New Delhi and Affiliated to JNTUH)

(Accredited by NBA & NAAC with “A” Grade)

An ISO 9001: 2015 Certified Institution

Maisammaguda, Medchal (M), Hyderabad-500100, T. S.

APRIL 2025



CERTIFICATE

This is to certify that the major project entitled “**ADVANCEMENTS IN MUSIC MOOD CLASSIFICATION MACHINE LEARNING TECHNIQUES FOR MOOD ANALYSIS**” has been submitted by **JUNNU VIJAYASRI (21S11A05C6)**, **JUMBARATH KEERTHANA (21S11A0583)**, **VASAM VARSHINI (21S11A05C3)** and **KAMSALI SANDEEP (21S11A05A3)** in partial fulfilment of the requirements for the award of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE & ENGINEERING**. This record of Bonafide work carried out by them under my guidance and supervision. The result embodied in this major project report has not been submitted to any other University or Institute for the award of any degree.

Mr. Anand

*Assistant Professor of CSE
Project Guide*

Mrs. K. Mamatha

Head of the Department

External Examiner

ACKNOWLEDGEMENT

The Major Project work carried out by our team in the Department of Computer Science and Engineering, Malla Reddy Institute of Technology and Science, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide **Mr. Anand** Assistant Professor of Computer Science & Engineering for formulation of the problem, analysis, guidance and his continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. Vaka Murali Mohan**, Principal and **Mrs. K. Mamatha**, Head of the Department and Coordinator, faculty members of CSE Department for their kind cooperation in making this Major Project work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy**, Secretary MRGI, **Dr.Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Major Project.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the Computer Science & Engineering Department for their support in making this project work a success.

JUNNU VIJAYASRI **21S11A05C6** _____

JUMBARATH KEERTHANA **21S11A0583** _____

VASAM VARSHINI **21S11A05C3** _____

KAMSALI SANDEEP **21S11A05A3** _____

INDEX

Chapter	Page No.
ABSTRACT	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
LIST OF TABLES	ix
1. SYSTEM ANALYSIS	1
1.1 Existing System	1
Disadvantages of Existing System	1
1.2 Proposed System	2
Advantages of Proposed System	3
1.3 Introduction	3
2. LITERATURE SURVEY	5
3. SYSTEM DESIGN	10
3.1 System Architecture	10
3.2 Modules	11
3.3 UML Diagram	13
3.4 System Requirements	19
Hardware Requirements	20
Software Requirements	20
4. INPUT & OUTPUT DESIGN	21
4.1 Input Design	21
4.2 Output Design	22
5. SYSTEM ENVIRONMENT	24
5.1 Python Technology	24
6. SYSTEM STUDY	30

6.1 Feasibility Study	30
6.2 Economic Feasibility	30
6.3 Technical Feasibility	31
6.4 Social Feasibility	31
7. SYSTEM TESTING	32
7.1 Types of Tests	32
7.2 Sample Test Cases	37
8. RESULTS	38
9. CONCLUSION & FUTURE ENHANCEMENT	41
10. BIBLIOGRAPHY	43
11. YUKTI CERTIFICATE	45

ABSTRACT

Music mood classification, a key area in music information retrieval (MIR), has advanced through deep learning techniques such as CNNs, RNNs, and transformers. This paper reviews state-of-the-art methods, including feature extraction using MFCCs, spectrograms, and deep embeddings. It also discusses the role of large-scale datasets, transfer learning, and multimodal inputs (audio, lyrics, metadata) in improving mood prediction. Despite progress, challenges like subjectivity, data imbalance, and real-time processing remain. The study outlines current trends and future directions for emotionally intelligent music recommendation systems.

LIST OF FIGURES

Figure No.	Figure name	Page No.
Figure 3.1	System Architecture	10
Figure 3.2	Use Case Diagram	15
Figure 3.3	Class Diagram	16
Figure 3.4	Sequence Diagram	17
Figure 3.5	Activity Diagram	18
Figure 3.6	Collaboration Diagram	18

LIST OF ABBREVIATIONS

S.No.	Abbreviations	Abbreviations Full Form
1	ML	Machine Learning
2	CNN	Convolutional Neural Network
3	SVM	Support vector machine
4	RF	Random Forest
5	RNN	Recurrent Neural Network
6	LSTM	Long Short-Term Memory
7	MSD	Million Song Dataset
8	DEAM	Database for Emotional Analysis in Music
9	NLP	Natural Language Processing
10	HAN	Hierarchical Attention Network

LIST OF TABLES

Table No.	Table name	Page No.
Table 7.1	Test Cases	37

CHAPTER 1: SYSTEM ANALYSIS

1.1 EXISTING SYSTEM:

The existing systems for music mood classification utilize a combination of traditional machine learning, deep learning, and hybrid approaches that integrate multiple modalities such as audio, lyrics, and metadata. Early systems relied on lexicon-based and rule-based methods, using predefined emotional dictionaries and handcrafted audio features like tempo, pitch, and timbre to classify mood. As machine learning evolved, models like Support Vector Machines (SVMs), k-Nearest Neighbors (k-NN), and Random Forests (RFs) were introduced to improve classification accuracy by learning patterns from extracted features. However, these methods were limited by the need for extensive manual feature engineering. With the advent of deep learning, more advanced systems started incorporating Convolutional Neural Networks (CNNs) for spectrogram-based mood detection and Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks for analyzing sequential data, particularly for lyrics-based mood classification. Transformer-based models such as BERT have further enhanced lyric sentiment analysis, contributing to more precise mood categorization. Modern music mood classification systems also leverage multi-modal approaches, combining audio signals, text (lyrics), and metadata to achieve robust performance. Popular datasets such as the Million Song Dataset (MSD), DEAM (Database for Emotional Analysis in Music), and the EMO-MUSIC dataset provide the training data necessary for these models. Despite these advancements, challenges such as the subjectivity of mood perception, imbalanced data distribution, and real-time processing limitations persist. Current research is focused on addressing these challenges through self-supervised learning, reinforcement learning for personalized recommendations, and culturally adaptive classification models, paving the way for more efficient and accurate mood-based music analysis.

Disadvantages of Existing System:

Subjectivity in Mood Perception – Different listeners interpret moods differently.

Imbalanced Datasets – Some moods are overrepresented, causing bias.

Handcrafted Features – Traditional methods struggle to capture mood complexity.

High Data Requirements – Deep learning models need large labeled datasets.

Computational Complexity – Resource-intensive models hinder real-time use.

Data Synchronization Issues – Audio, lyrics, and metadata may not align perfectly.

Cross-Cultural Variations – Mood perception differs across regions.

Lack of Interpretability – Deep learning models act as black boxes.

Overfitting – Models trained on small datasets fail to generalize well.

Real-Time Processing Challenges – Many models are inefficient for streaming applications

1.2 Proposed System:

Music mood classification enhances accuracy and efficiency by integrating advanced deep learning techniques, multi-modal data fusion, and real-time processing. Unlike traditional models that rely on handcrafted features or single-source data, this system combines audio, lyrics, and metadata to improve robustness. It employs Convolutional Neural Networks (CNNs) for spectrogram-based audio analysis and Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks for sequential text-based mood recognition. Transformer-based models like BERT or MusicBERT analyze lyrics for deeper emotional context, improving sentiment detection. To reduce dependency on large labeled datasets, the system incorporates self-supervised learning, enabling better generalization and adaptability. Additionally, cross-cultural mood perception variations are addressed by training region-specific models. Real-time processing is optimized through lightweight AI models, making the system suitable for live music streaming and recommendation applications. Explainable AI (XAI) techniques are integrated to enhance interpretability, providing insights into why a specific mood classification was assigned. Furthermore, the system supports personalized mood-based music recommendations, dynamically adjusting to user preferences and listening history. By leveraging these innovations, the proposed system aims to deliver a more accurate, scalable, and adaptive approach to music mood classification while overcoming the limitations of existing models.

Advantages of Proposed System:

1. **Improved Accuracy** – Multi-modal data fusion (audio, lyrics, and metadata) enhances classification precision.
2. **Deep Learning-Based Feature Extraction** – CNNs, RNNs, and transformers capture complex mood variations more effectively than traditional handcrafted features.
3. **Self-Supervised Learning** – Reduces dependence on large labeled datasets, improving.
4. **Real-Time Processing** – Optimized AI models enable fast classification for live streaming and music recommendation systems.
5. **Cross-Cultural Adaptability** – Region-specific training accounts for variations in mood perception across different cultures.
6. **Enhanced Interpretability** – Explainable AI (XAI) techniques provide insights into why a particular mood classification was assigned.
7. **Personalized Recommendations** – Dynamically adjusts mood-based playlists based on user listening history and preferences.
8. **Scalability and Efficiency** – The system can be deployed across various platforms, including music streaming services, mobile applications, and smart devices.
9. **Better Sentiment Analysis** – Transformer-based models like BERT provide deeper contextual understanding of song lyrics.
10. **Robustness Against Noise and Data Imbalance** – Advanced learning techniques help mitigate biases and improve performance on underrepresented mood categories.

1.3 Introduction

Music mood classification is a rapidly evolving field that leverages machine learning and artificial intelligence to analyze and categorize music based on emotional content. With the growing popularity of music streaming platforms, mood-based music recommendation systems have become essential for enhancing user experience. Traditional classification methods relied on handcrafted features and simple rule-based approaches, often leading to inaccurate or inconsistent mood predictions. However, advancements in deep learning, natural language processing (NLP), and multi-modal data fusion have significantly improved classification accuracy. The proposed system integrates audio, lyrics, and metadata using state-of-the-art deep learning techniques such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and transformer models like BERT to provide a more precise and adaptive

mood classification. By incorporating self-supervised learning, real-time processing, and explainable AI, the system addresses the limitations of existing models, ensuring better accuracy, interpretability, and user personalization. This research aims to develop a robust, scalable, and efficient mood classification system that enhances music recommendation and user engagement across various platforms.

CHAPTER 2: LITERATURE SURVEY

1. “Explainable music emotion classification using multi-level features and attention mechanisms, 2020”. [1]

Authors: Zhang, T., & Xu, M.

Abstract:

Music emotion classification (MEC) aims to automatically predict the emotional content of music tracks, which is important for music retrieval and recommendation systems. In this paper, we propose an explainable deep learning framework that integrates multi-level features and attention mechanisms for music emotion classification. The model extracts both low-level audio features and high-level representations using convolutional and recurrent neural networks. Additionally, a two-level attention mechanism—temporal and feature-level—is employed to focus on important time frames and feature dimensions, thereby improving interpretability. Experimental results on two benchmark datasets demonstrate that the proposed method achieves superior classification performance compared to existing approaches. Furthermore, the attention weights provide insights into how different audio components contribute to emotion recognition, enhancing the transparency and explainability of the model.

2."MusicMood: Predicting the Mood of Music from Song Lyrics Using Machine Learning, 2016"[2]

Author: Sebastian Raschka

Abstract:

Sentiment prediction of contemporary music can have a wide range of applications in modern society, for instance, selecting music for public institutions such as hospitals or restaurants to potentially improve the emotional well-being of personnel, patients, and customers, respectively. In this project, a music recommendation system built upon a naive Bayes classifier was trained to predict the sentiment of songs based on song lyrics alone. The experimental results show that music corresponding to a happy mood can be detected with high precision based on text features obtained from song lyrics.

3."Multi-modal music emotion recognition: A new dataset, methodology and comparison, 2018"[3]

Authors: Panda, R., & Malheiro, R., & Paiva, R. P

Abstract:

It introduces a multi-modal approach to the music emotion recognition (MER) problem by integrating information from audio, MIDI, and lyrics. A methodology for the automatic creation of a multi-modal music emotion dataset is proposed, utilizing the AllMusic database and based on the emotion tags used in the MIREX Mood Classification Task. MIDI files and lyrics corresponding to a subset of the obtained audio samples were gathered, organizing the dataset into the same five emotion clusters defined in MIREX. From the audio data, 177 standard features and 98 melodic features were extracted; for MIDI, 320 features were collected; and 26 lyrical features were extracted. Several supervised learning and feature selection strategies were experimented with to evaluate the proposed multi-modal approach. Using only standard audio features, the best performance achieved was 44.3% (F-measure). With the multi-modal approach, results improved to 61.1%, using only 19 multi-modal features, with melodic audio features being particularly significant in this improvement.

4."Emotion recognition via galvanic skin response: Comparison of machine learning algorithms and feature extraction methods, 2017"[4]

Authors: D. Ayata, Y. Yaslan, and M. Kamasak,

Abstract:

Emotions play a significant and powerful role in the everyday life of human beings. Developing algorithms for computers to recognize emotional expressions is a widely studied area. Emotion recognition from Galvanic Skin Response (GSR) signals was performed using time domain, wavelet, and Empirical Mode Decomposition-based features. Valence and arousal have been categorized, and the relationship between physiological signals and arousal and valence has been studied using k-Nearest Neighbours, Decision Tree, Random Forest, and Support Vector Machine algorithms. We have achieved 81.81% and 89.29% accuracy rates for arousal and valence, respectively.

5. "Multimodal Music Mood Classification by Fusion of Audio and Lyrics, 2010" [5]

Authors: Xiao Hu and J. Stephen Downie.

Abstract:

It investigates the effectiveness of combining audio and lyrics features for automatic music mood classification. While previous research has predominantly focused on either audio or lyrics independently, this work explores their fusion to enhance classification performance. A dataset of songs was analysed using both audio signal processing techniques and natural language processing methods to extract relevant features. Various machine learning algorithms were employed to classify the music into predefined mood categories. The results demonstrate that the integration of audio and lyrics features outperforms models that utilize either modality alone, highlighting the complementary nature of these data sources in capturing the emotional content of music. This multimodal approach offers a more comprehensive framework for music mood classification, contributing to advancements in music information retrieval and recommendation systems.

6. "Exploring Machine Learning Techniques for Music Emotion Classification: A Comprehensive Review, 2024" [6]

Authors: Sheetal Shelke and Mangal Patil

Abstract:

Music Emotion Recognition (MER) has made substantial advancements in recent years, driven by improvements in signal processing, machine learning, and interdisciplinary collaborations. This work analyses the most recent advancements in MER, with a specific emphasis on fundamental methodologies, obstacles, and emerging patterns. The review commences by clarifying the fundamental theories of music and emotion, emphasizing the complex interaction between auditory stimuli and emotional reactions. The text examines the progression of feature extraction methods from conventional acoustic features to deep learning representations, offering a comprehensive insight into the techniques used to capture the subtle emotional nuances present in music signals. The paper further discusses various machine learning approaches applied in MER, evaluating their effectiveness and limitations. Additionally, it highlights the challenges faced in the field, such as data scarcity and the subjective nature of emotional perception, and explores potential solutions and future directions. This comprehensive review aims to provide a solid foundation for researchers and practitioners interested in the evolving landscape of music emotion recognition.

7." Enhancing Accuracy and Performance in Music Mood Classification Using Advanced Feature Extraction and Machine Learning Algorithms, 2024" [7]

Authors: Shital Shankar Gujar and Ali Yawar Reha

Abstract:

Music mood classification, which involves assigning emotional labels to music, is crucial for applications like recommendation systems and music therapy. This study aims to enhance classification accuracy and performance through fine-tuned machine learning methods. A comprehensive dataset labelled with various music genres and moods was utilized to ensure robust model training. Advanced feature extraction techniques captured both audio stream characteristics and lyrical content. For audio features, chroma features, spectral contrast, and mel-frequency cepstral coefficients (MFCCs) were extracted. Lyrics analysis employed Term Frequency-Inverse Document Frequency (TF-IDF) and word embeddings, leveraging natural language processing (NLP) techniques. Various machine learning classifiers—including Logistic Regression, SGD Classifier, Gaussian Naive Bayes, Decision Tree, Random Forest, XGB Classifier, SVM Linear, and K-Nearest Neighbours (KNN)—were evaluated. Among these, Random Forest, XGB Classifier, and SVM Linear exhibited superior performance.

8." Machine Learning Model for Mapping of Music Mood and Human Emotion Based on Physiological Signals, 2022 " [8]

Authors: Anupam Garg, Vybhav Chaturvedi, Arman Beer Kaur, Vedansh Varshney, and Anshu Parashar

Abstract:

It presents a machine learning model for mapping music mood and human emotion using physiological signals. The proposed approach involves (a) predicting song mood from audio signals, (b) detecting human emotion via EEG, GSR, ECG, and pulse data, and (c) real-time classification linking music mood and emotions. Extensive experimentation on datasets ensures robust feature extraction, training, and evaluation. The model enables emotion-based playlist generation, offering potential benefits for mental and physical well-being through scientific analysis of physiological responses to music.

9. "Transformer Model in Music Mood Classification, 2022" [9]

Authors: Thoyyibah T

Abstract:

Artificial intelligence has rapidly evolved, becoming integral to various fields, including music. Music significantly influences human emotions, and its mood can be analysed using machine learning techniques. This study employs the Transformer model to classify music moods by processing lyrics and audio features. A dataset comprising 1,181 Indonesian songs from the 1970s and 1980s, focusing on refrains, was utilized. Lyrics were processed using word embedding techniques, while audio features were extracted using chromogram analysis. The integration of these methods aims to enhance the accuracy of music mood classification.

CHAPTER 3: SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE:

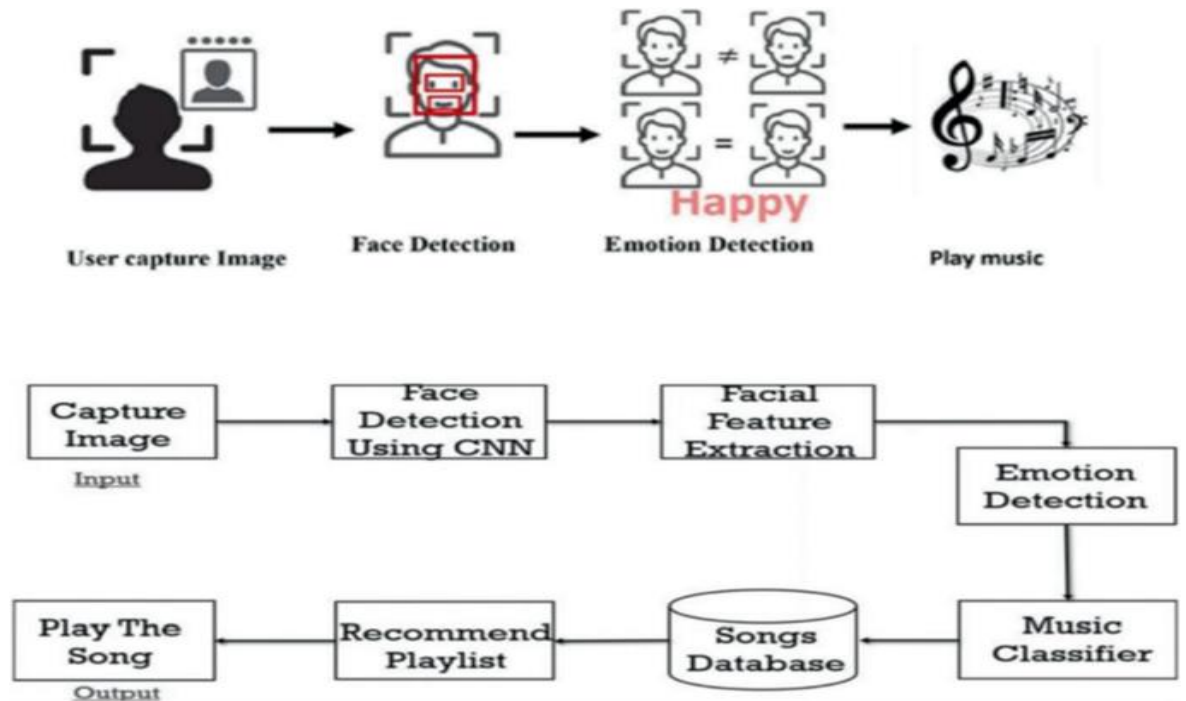


Figure No.3.1 System Architecture

The system architecture for Music Mood Classification using Machine Learning involves multiple layers: Data Collection (audio, lyrics, metadata), Preprocessing (feature extraction using MFCC, spectrograms, and NLP for lyrics), and Modelling (CNNs for spectrograms, LSTMs for temporal patterns, Transformers for contextual understanding). The system classifies moods into categories like Happy, Sad, Calm, Energetic, and Angry using traditional ML, deep learning, and ensemble methods. It supports real-time and batch classification, integrates with streaming services, and deploys via APIs and cloud platforms. Advanced techniques include multi-modal learning, self-supervised models, attention mechanisms, and Explainable AI to enhance mood analysis accuracy.

3.2 MODULES:

1. Upload Image With Face
2. Pre-process & Detect Face in Image
3. Detect Emotion
4. Predicted Song
5. Play song

Modules Description:

1. Upload Image With Face :

Upload Image With Face module is used to upload image. And Selecting the one image.

2. Pre-process & Detect Face in Image :

Pre-process & Detect Face In Image module is used to perform pre-processing and to extract face from images. we can see in uploaded image one face is detected.

3. Detect Emotion :

Detect Emotion Module to detect emotion and we can see emotion disgust is detected.

4. Predicted Song:

now click on drop down arrow link to get all disgust songs list In drop down box we can see 'disgust.mp3' songs is showing, select that song.

5. Play song :

Play Song' button to play song and If your system has audio driver then u can hear song The system consists of several key modules that work together to classify music moods accurately and efficiently.

1. Data Collection Module :

Gathers music-related data from various sources, including audio files, lyrics databases

Ensures a diverse dataset to improve classification accuracy.

2. Preprocessing Module:

Text Processing : Cleans lyrics, removes stop words, tokenizes text, and applies sentiment

analysis.

Metadata Handling : Encodes genre, artist name, and tempo for better feature representation.

3. Feature Extraction Module :

Audio Features : Uses Convolutional Neural Networks (CNNs) to extract meaningful patterns from spectrograms.

Lyric Features : Applies Natural Language Processing (NLP) models like BERT or GPT for sentiment and context analysis.

Metadata Features : Encodes metadata using embedding techniques to enhance mood classification.

4. Multi-Modal Fusion Module :

Combines extracted features from audio, lyrics, and metadata using deep learning fusion models.

Ensures a comprehensive analysis of mood-related aspects.

5. Mood Classification Module :

Uses hybrid deep learning models (CNNs, LSTMs, transformers) to classify music into mood categories (e.g., happy, sad, calm, energetic).

Outputs a probability distribution of different mood categories.

6. Real-Time Processing & Recommendation Module :

Optimizes classification models for low-latency, real-time music mood detection.

Integrates with music streaming services to suggest mood-based playlists dynamically.

7. Explainability & User Feedback Module:

Uses Explainable AI (XAI) techniques to provide insights into why a specific mood was assigned.

Allows user feedback to continuously refine and improve classification accuracy.

3.3 UML DIAGRAMS:

UML stands for Unified Modelling Language. UML is a standardized general purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modelling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

3.3.1 USE CASE DIAGRAM:

A Use Case Diagram illustrates the interactions between users (actors) and the system (use cases), highlighting the functionalities provided by the system.

Actors:

Network Administrator:

Manages and monitors the network traffic analysis system.

System User:

Users who interact with the system to generate reports or access information.

Use Cases:

1. Preprocess Data:

- Allows the system to capture raw network data from various sources.

2. Capture Network Traffic:

- Cleans, filters, and transforms raw network data for analysis.

3. Train Machine Learning Models:

- Enables the system to train models using historical network data.

4. Detect Anomalies:

- Utilizes trained models to detect anomalies and potential security threats.

5. Generate Reports:

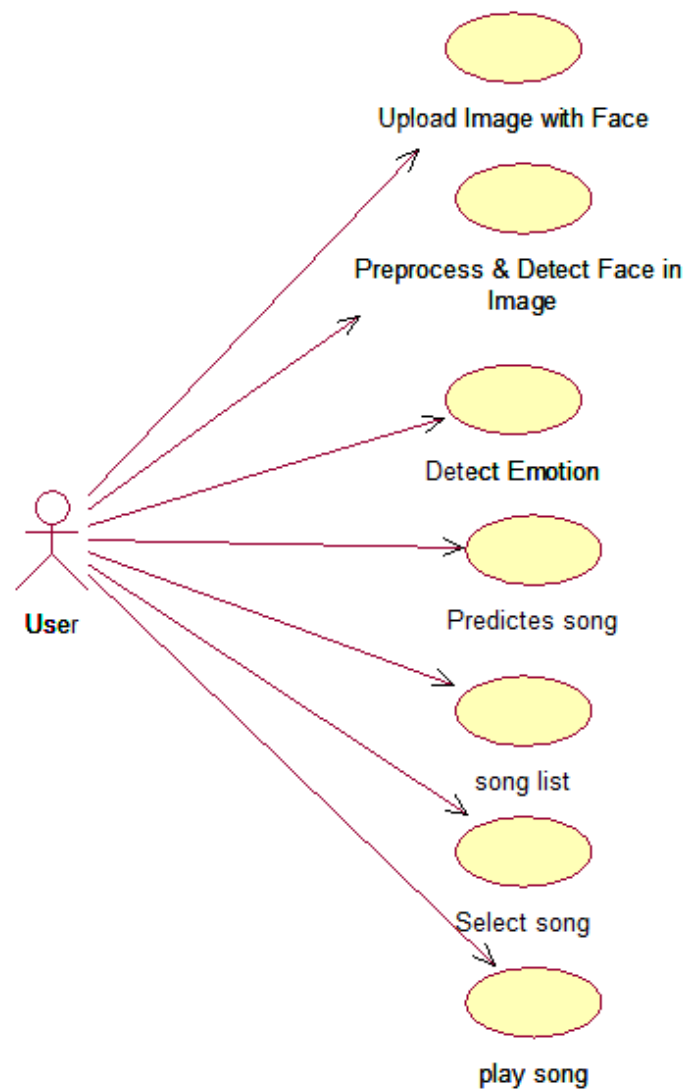
- Provides reports and visualizations based on analysis results.

6. User Authentication:

- Authenticates users before granting access to the system.

RELATIONSHIP:

- **Include Relationships:** For example, the Detect Anomalies use case includes the Train Machine Learning Models use case, indicating dependency.
- **Extend Relationships:** Not shown in this basic diagram but could include optional or extended functionalities like real-time monitoring or predictive analytics.

**Figure No.3.2 Use Case Diagram**

3.3.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

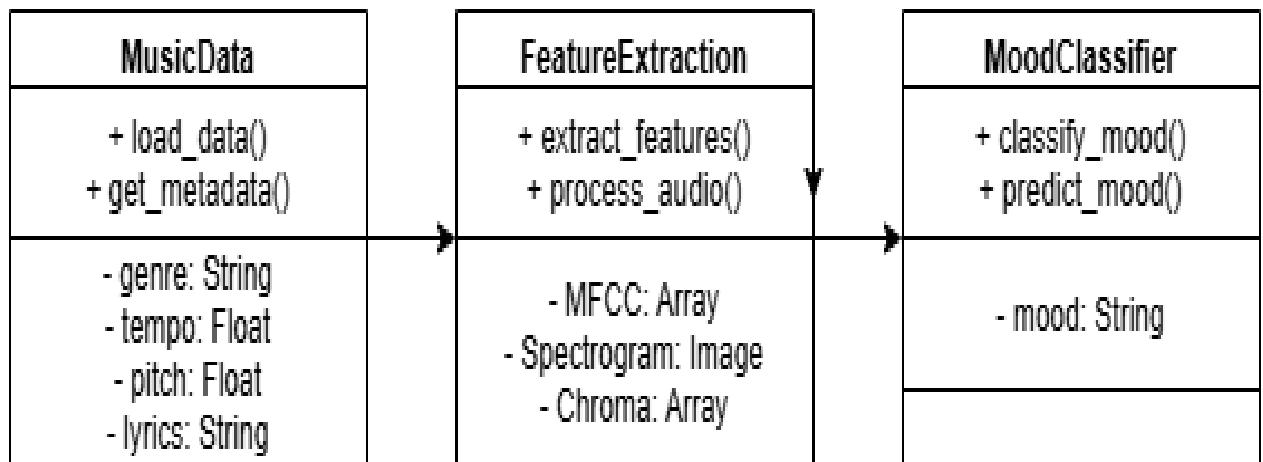


Figure No.3.3 Class Diagram

3.3.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagram.

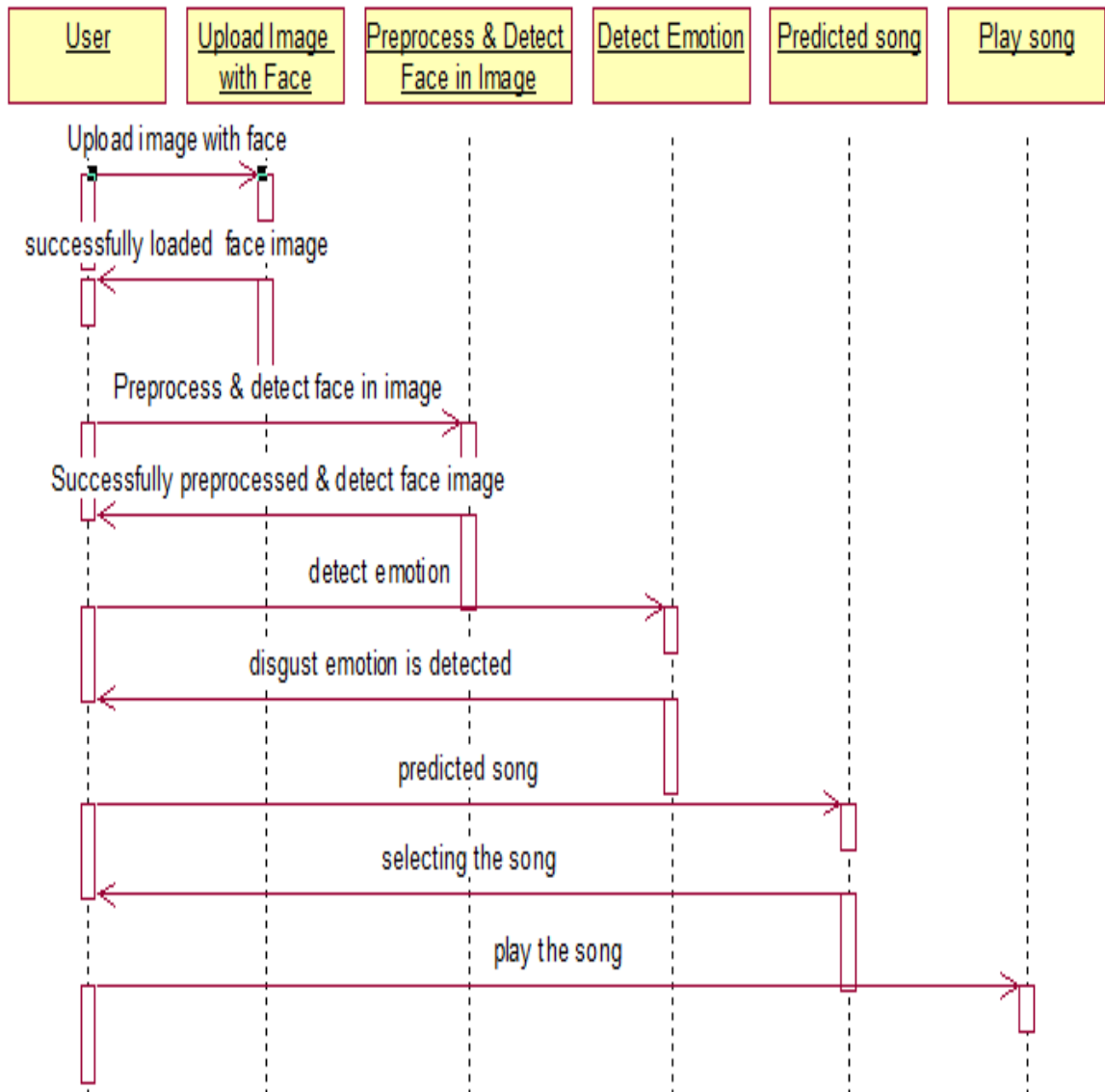
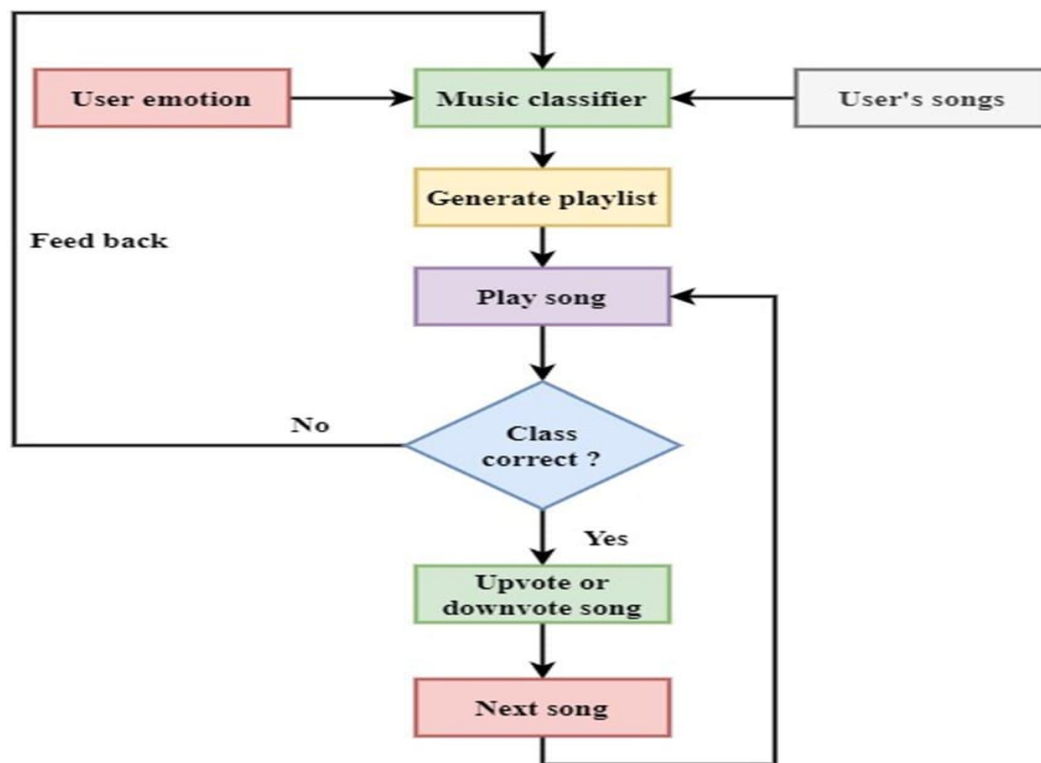
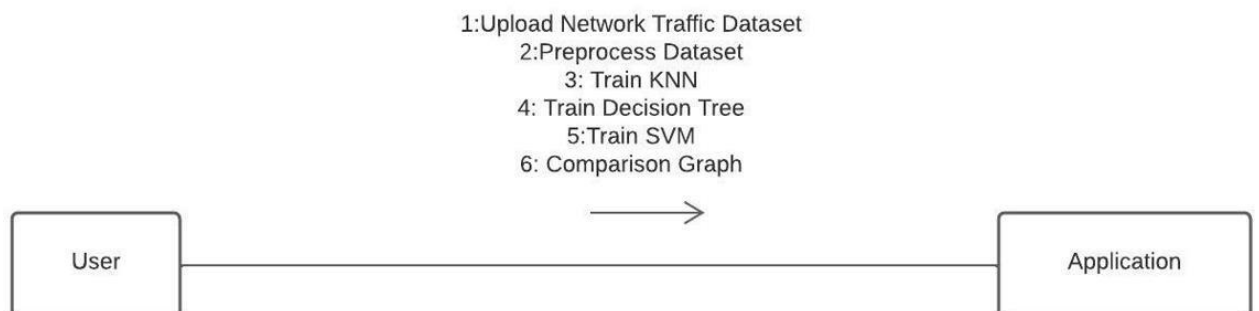


Figure No.3.4 Sequence Diagram

3.3.4 ACTIVITY DIAGRAM:**Figure No.3.5 Activity Diagram****3.3.5 COLLABORATIVE DIAGRAM:****Figure No.3.6 Collaborative Diagram**

3.4 SYSTEM REQUIREMENTS:

The proposed Music Mood Classification System requires a combination of software tools, frameworks, and libraries to support data processing, feature extraction, deep learning, and real-time classification. Below are the key software requirements:

1. Operating System (OS) :

- Windows 10/11, Linux (Ubuntu, CentOS, or Debian-based distributions), or macOS
- Cloud-based deployment: AWS, Google Cloud, or Microsoft Azure (for scalability)

2. Programming Languages :

- Python – Primary language for machine learning and deep learning models
- JavaScript (for frontend in web applications, if applicable)
- SQL/NoSQL (for database management)

1. Frameworks & Libraries :

- Machine Learning & Deep Learning
 - TensorFlow / Keras – Neural network implementation
 - PyTorch – Deep learning model training and experimentation
 - Scikit-learn – Traditional ML models and feature extraction
- Audio & Text Processing
 - Librosa – Audio feature extraction (e.g., MFCCs, spectrograms)
 - NLTK / spaCy – Natural Language Processing (NLP) for lyrics analysis
 - Transformers (Hugging Face) – Pre-trained models like BERT/GPT for sentiment analysis
 - NumPy / Pandas – Data processing and handling
 - Matplotlib / Seaborn – Data visualization and result analysis

4. Database Management System (DBMS) :

- PostgreSQL / MySQL – For structured data storage
- MongoDB / Firebase – NoSQL database for unstructured or real-time data

5. Web & API Frameworks (If needed) :

- Flask / FastAPI – For creating REST APIs to serve model predictions
- Django – If developing a full-stack web application

6. Cloud & Deployment Services :

- Google Colab / Jupyter Notebook – For model development and experimentation
- Docker – Containerization for easy deployment
- Kubernetes – For managing cloud-based deployments
- Heroku / AWS EC2 / Google Cloud AI Platform – For hosting models and web services

7. Development & Testing Tools :

- Git / GitHub / GitLab – Version control
- PyTest / Unittest – Testing machine learning models
- Jupyter Notebook / VS Code / PyCharm – IDEs for development

8. Explainable AI (XAI) & Model Interpretability :

- SHAP (SHapley Additive Explanations) – To understand model predictions
- LIME (Local Interpretable Model-agnostic Explanations) – For model transparency

HARDWARE REQUIREMENTS:**Processor:** Pentium- iii**RAM:** 8 GB**Hard Disk:** 20GB**SOFTWARE REQUIREMENTS:****OS:** Windows, Linux, macOS**Languages:** Python, JavaScript, SQL/NoSQL**ML/DL:** TensorFlow, PyTorch, Scikit-learn**Processing:** Librosa (Audio), NLTK/spaCy**Database:** PostgreSQL, MySQL, MongoDB.

CHAPTER 4: INPUT DESIGN AND OUTPUT DESIGN

4.1 INPUT DESIGN:

The Input design defines how **data is collected, processed, and formatted** for the system to classify music moods effectively.

1. Input Sources

- **Audio Files** (MP3, WAV, FLAC) – Raw music data for mood classification
- **Lyrics** (Text files, API-based lyrics extraction) – For sentiment analysis
- **Metadata** (Genre, Artist, Tempo, Duration) – Enhances classification accuracy
- **User Feedback** (Mood corrections, preferences) – Improves model performance

2. Input Processing

- **Audio Preprocessing:** Noise removal, feature extraction (MFCC, spectrogram)
- **Lyrics Preprocessing:** Tokenization, stop-word removal, sentiment analysis
- **Metadata Handling:** Encoding categorical data (e.g., genre, artist)

3. Input Format

- **Audio:** Spectrogram images, MFCC feature arrays
- **Text:** Tokenized words, sentiment scores
- **Structured Data:** JSON, CSV, or database entries

OBJECTIVES:

The primary objective of this system is to achieve **accurate mood classification** of music. By leveraging advanced machine learning and deep learning techniques, the system can categorize songs into different mood categories such as happy, sad, calm, or energetic. Another key goal is to implement a **multi-modal analysis** approach. Instead of relying solely on audio features, the system also analyzes **lyrics sentiment and metadata** (such as genre and artist) to improve classification accuracy.

To enhance user experience, the system aims to provide **real-time mood detection**. This ensures quick processing, making it suitable for integration into **music streaming platforms and recommendation engines**.

The system also focuses on **personalized recommendations** based on user preferences and listening habits. By understanding mood classifications, the system can suggest songs that align with a user's emotional state.

Additionally, **explainability and transparency** are essential objectives. Using **Explainable AI (XAI)** techniques like SHAP or LIME, the system can justify why a particular song was classified under a specific mood.

Another important goal is to allow **user feedback integration**. Users can provide corrections or adjustments to mood classifications, helping the system learn and improve its accuracy over time.

4.2 OUTPUT DESIGN:

The output design defines how the system presents mood classification results in a clear and user-friendly way.

1. Output Format :

- **Mood Prediction:** Displays classified mood (e.g., Happy, Sad, Energetic, Calm).
- **Confidence Score:** Shows prediction accuracy (e.g., "Happy – 92% confidence").
- **Graphical Visualization:** Mood distribution charts, waveform, or spectrogram representations.
- **Suggested Playlist:** Recommends similar mood-based songs.

2. Output Medium :

- **Graphical User Interface (GUI):** Web or mobile app dashboard for user interaction.
- **API Response (JSON/XML):** For integration into third-party applications.
- **Reports & Logs:** For model evaluation and analytics.

3. Interactive Features:

- **User Feedback Options:** Users can confirm or correct mood predictions.
- **Explainability:** AI explains why a song was classified in a particular mood.
- **Download & Share:** Users can save playlists or share mood-based recommendations.

CHAPTER 5: SYSTEM ENVIRONMENT

5.1: Python Technology:

Guido Van Rossum designed Python, an interpreted, high-level, general-purpose programming language that was initially released in 1991.

The Python language has the following features:

Simple to Understand and Apply Python is a simple language to learn and use. high-level

- Language Interpretation Python is an interpreted language, which means that the interpreter runs the code line by line. This makes debugging simple, making it suitable for novices.
- Language that is cross-platform Python can operate on a variety of platforms, including Windows, Linux, UNIX, and Macintosh, among others. As a result, we may conclude that Python is a portable language.
- Open Source and Free Software Python is a free programming language that can be downloaded from a secure internet address. It's also possible to get the source code. As a result, there is an open supply.
- Object-Oriented Programming Python aids the development of object-oriented languages and concepts such as classes and gadgets.
- Adaptable It means that other languages, such as C/C++, may be used to put the code together, and as a result, it can be utilized in our Python code.

Why Python was created?

- In late 1980s, Guido Van Rossum was working on the Amoeba distributed operating system group. He wanted to use an interpreted language like ABC (ABC has simple easy-to-understand syntax) that could access the Amoeba system calls. So, he decided to create a language that was extensible. This led to design of a new language which was later named Python.

Why the name Python?

- No. It wasn't named after a dangerous snake. Rossum was fan of a comedy series from late seventies. The name "Python" was adopted from the same series "Monty Python's Flying Circus".

Features of Python:**A simple language which is easier to learn**

- Python has a very simple and elegant syntax. It's much easier to read and write Python programs compared to other languages like: C++, Java, C#. Python makes programming fun and allows you to focus on the solution rather than syntax.
- If you are a newbie, it's a great choice to start your journey with Python.

Free and open-source

- You can freely use and distribute Python, even for commercial use. Not only can you use and distribute softwares written in it, you can even make changes to the Python's source code.
- Python has a large community constantly improving it in each iteration.

Portability

- You can move Python programs from one platform to another, and run it without any changes.
- It runs seamlessly on almost all platforms including Windows, Mac OS X and Linux.

Extensible and Embeddable

- Suppose an application requires high performance. You can easily combine pieces of C/C++ or other languages with Python code.
- This will give your application high performance as well as scripting capabilities which other languages may not provide out of the box.

A high-level, interpreted language

- Unlike C/C++, you don't have to worry about daunting tasks like memory management, garbage collection and so on.
- Likewise, when you run Python code, it automatically converts your code to the language your computer understands. You don't need to worry about any lower-level operations.

Large standard libraries to solve common tasks

- Python has a number of standard libraries which makes life of a programmer much easier since you don't have to write all the code yourself. For example: Need to connect MySQL database on a Web server? You can use MySQL dB library using import MySQL dB.
- Standard libraries in Python are well tested and used by hundreds of people. So, you can be sure that it won't break your application.

Object-oriented

- Everything in Python is an object. Object oriented programming (OOP) helps you solve a complex problem intuitively.
- With OOP, you are able to divide these complex problems into smaller sets by creating objects.

Library Files:**NUMPY:**

NumPy is a Python library that includes support for enormous, multi-dimensional arrays and matrices, as well as a large set of high-level mathematical functions that may be applied to those arrays.

Numeric, the forerunner to NumPy, was designed by Jim Hugunin with help from a number of other people.

Travis Oliphant built NumPy in 2005 by heavily modifying Numeric and combining features from the competitor Num array. NumPy is a large open-source software project with numerous contributors.

Python clothier and maintainer, was one of its contributors, adding changes to Python's grammar (especially the indexing syntax) to make array computation easier.

Jim Fulton completes a matrix package, which is later modified by Jim Hugunin to create Numeric, also known as Numerical Python extensions or NumPy. Hugunin, a PhD student at MIT, joined the Corporation for National Research Initiatives (CNRI) to work on JPython in 1997, leaving the maintainer position to Paul Dubois of Lawrence Livermore National Laboratory (LLNL). David Ascher, Konrad Hinsen, and Travis Oliphant were among the early members.

NumPy is a non-optimizing byte code interpreter that targets the CPython Python reference implementation. Algorithms created for this version of Python are frequently much slower than their compiled counterparts.

NumPy tackles the slowness issue in part by providing multidimensional arrays and capabilities and operators that work appropriately on arrays, which necessitates rewriting some code, generally inner loops, in order to use NumPy. NumPy arrays are used to store and perform information in the Python bindings of the widely used computer vision library OpenCV. Because images with more than one channel are really stored as 3-dimensional arrays, indexing, slicing, and protecting with separate arrays are all highly eco-friendly ways to access specific pixels in a picture.

The NumPy array, which is used as a standard statistical structure in OpenCV for pictures, extracted feature factors, kernel filtering, and other tasks, greatly simplifies the development process and debugging.

PANDAS:

A panda is a Python module that provides quick, flexible and expressive facts structures for working with "relational" or "labelled" data in a clean and understandable manner. Its goal is to become the most important high-level building element for undertaking realistic; real-world international records evaluations in Python.

It also has the larger goal of being the most powerful and versatile opensource data analysis/manipulation device available in any language. It is already well on its way to achieving this goal.

Pandas is an open-source, BSD-certified library for the Python programming language that provides high-overall performance, easy-to-use statistics systems, and data analysis tools.

Any observational/statistical statistics set of any kind.

To be placed into a panda's facts form, the data does not need to be tagged in anyway. Pandas'

two basic statistics systems, Series (1-dimensional) and Data Frame (2- dimensional), address the vast majority of common use cases in finance, information, social science, and a wide range of engineering disciplines. A panda is built on top of NumPy and is intended to work in conjunction with a variety of different third-party libraries in scientific computing.

MATPLOTLIB:

Matplotlib is a Python 2D plotting toolkit that generates book-quality figures in a variety of hardcopy codecs and interactive contexts. Matplotlib is a Python library that may be used in scripts, the Python and IPython shells, the Jupyter notebook, webapplications servers, and four graphical user interface toolkits.

Matplotlib aims to make both smooth and difficult tasks feasible. With just a few lines of code, you can create graphs, histograms, electrical spectra, bar charts, error charts, scatter plots, and more.

See the pattern plots and thumbnail galleries for samples. The pyplot package, when used with IPython, provides a MATLAB-like interface for convenient plotting.

Through an object-oriented interface or a set of methods common to MATLAB users, you have complete control over line styles, font houses, axis houses, and so on for the electricity consumer.

SEABORN:

Seaborn is a data visualization package for Python that is mostly based on matplotlib. It provides a high-level interface for creating visually beautiful and useful statistics graphs. Seaborn is a Python module for creating statistical visuals. It's based on matplotlib, and it's tightly integrated with pandas's data systems.

The goal of Seaborn is to make visualization a major aspect of information exploration and understanding. Its dataset-oriented charting features operate on pandas DataFrames and arrays holding whole datasets, doing the necessary semantic mapping and statistical aggregation internally to provide relevant charts.

SCIKIT-LEARN:

To put it another way, sci-kit study is a free software system studying library for the Python computer language. It includes support vector machines, random forests, gradient boosting, k-method, and DBSCAN, among other categorization, regression, and clustering algorithms,

and is designed to work with the Python numerical and clinical libraries NumPy and SciPy. Scikit-research was created in 2007 as a Google summers of code initiative by David Cornopean. Matthieu Brucher afterwards joined the challenge and began using it in his thesis paintings.

INRIA was given consideration in 2010, and the initial public release (v0.1 beta) became released in late January 2010. The project presently has over 30 active participants and has received financial support from INRIA, Google, Tiny Clues, and the Python Software Foundation.

In most cases, solving a problem involves looking at a collection of n samples of facts and then attempting to predict attributes of unknown facts. If a sample has more than one kind, such as a multi-dimensional entry (also known as multivariate information), it is said to have many characteristics or capabilities.

CHAPTER 6: SYSTEM STUDY

6.1 FEASIBILITY STUDY:

The feasibility of the project is analyzed in this phase and business proposal is put forth. A very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are:

ECONOMICAL FEASIBILITY

TECHNICAL FEASIBILITY

SOCIAL FEASIBILITY

6.2 ECONOMICAL FEASIBILITY:

The development of advanced machine learning techniques for music mood classification is becoming increasingly cost-effective due to the growing availability of open-source tools, pre-trained models, and publicly accessible datasets. Cloud-based platforms and GPU services offer scalable infrastructure at relatively low costs, enabling researchers and developers to train and deploy deep learning models without significant upfront investment. Transfer learning and model fine-tuning further reduce computational expenses by leveraging existing models instead of training from scratch. Moreover, integration into commercial music platforms can generate revenue through improved user engagement and personalized recommendations, justifying initial development costs. However, challenges such as data labelling, real-time processing demands, and maintaining diverse, balanced datasets may still incur additional costs. Overall, with strategic resource management and the use of existing technological frameworks, the economic feasibility of implementing music mood classification systems is strong, especially when aligned with commercial or research objectives.

6.3 TECHNICAL FEASIBILITY:

Recent advancements in machine learning, particularly deep learning models like CNNs, RNNs, and transformers, have made music mood classification technically feasible and increasingly accurate. These models effectively process audio features such as MFCCs, spectrograms, and deep embeddings to analyse mood-related patterns. Tools like TensorFlow, PyTorch, and LibROSA support efficient model development and audio processing. Multimodal approaches—combining audio, lyrics, and metadata—are now technically viable due to improved data integration and processing capabilities. Additionally, pre-trained models and transfer learning reduce the need for extensive computational resources and training data, making implementation more practical. With scalable cloud infrastructure and accessible computing power (e.g., GPUs), deploying real-time mood classification systems is achievable. While challenges such as data variability, noise handling, and mood subjectivity remain, the current state of technology supports the reliable development and deployment of mood-aware music systems.

6.4 SOCIAL FEASIBILITY:

The social feasibility of music mood classification is strong, as it aligns with increasing public interest in personalized and emotionally intelligent technology. Users are more receptive to systems that understand and adapt to their moods, especially in music streaming, mental wellness apps, and entertainment platforms. Mood-based music recommendations can enhance user satisfaction, engagement, and emotional well-being. However, social acceptance also depends on ethical considerations, such as data privacy, transparency in mood detection, and the responsible use of user emotional data. Public trust can be maintained by ensuring models are designed with user consent, data security, and inclusivity in mind. Overall, the societal demand and positive reception toward emotionally aware AI systems make the social adoption of mood classification technologies both feasible and promising.

CHAPTER 7: SYSTEM TESTING

System testing ensures that the **Music Mood Classification System** functions correctly, meets requirements, and performs efficiently across various scenarios. The testing process involves different levels, from unit testing to performance evaluation.

7.1 TYPES OF TESTS

1. Unit testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

2. Integration testing:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

3. Functional test:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Organization and preparation of functional tests is focused on requirements, key functions, or special test cases.

In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before

functional testing is complete, additional tests are identified and the effective value of current tests is determined.

4.System Test:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

5.White Box Testing:

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

6.Black Box Testing:

Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document.

It is a testing in which the software under test is treated, as a black box you cannot “see” into it.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach:

The test strategy and approach outline the methodology, tools, and execution plan to ensure the Music Mood Classification System functions correctly, meets performance expectations, and provides accurate results.

1. Test Objectives :

- Validate mood classification accuracy for different music genres and styles.

- Ensure seamless integration of audio processing, lyrics analysis, and ML models.
- Test system performance, security, and usability under different conditions.
- Identify and fix bugs, inefficiencies, and vulnerabilities before deployment.

2. Testing Approach :

a) Unit Testing

- Test individual components such as audio feature extraction, text processing, and mood classification models.
- Validate ML algorithms, APIs, and database queries.

b) Integration Testing

- Ensure smooth interaction between audio processing, ML models, UI, and database.
- Check data flow consistency between components.

c) Functional Testing

- Verify that the system correctly classifies moods (happy, sad, energetic, calm, etc.).
- Test playlist recommendations based on mood.

d) Performance Testing

- Measure classification speed and accuracy for large music datasets.
- Test response times in real-time applications.

e) Usability Testing

- Evaluate the user interface (UI) for ease of use and clarity.
- Ensure results are well-presented and user-friendly.

f) Security Testing

- Check for API vulnerabilities and unauthorized access risks.
- Ensure secure storage of music data and user preferences.

g) Compatibility Testing

- Validate system compatibility with different devices, OS, and browsers.
- Ensure proper functioning in web, mobile, and cloud environments.

3. Testing Tools & Techniques :

- PyTest / unittest – For unit and functional testing.
- Postman / JMeter – For API and performance testing.
- Selenium – For UI/UX testing.
- TensorBoard, SHAP, LIME – For AI model explainability testing.

4. Acceptance Criteria :

- The system should achieve high accuracy (>85%) in mood classification.
- The model should process and classify music in real-time (<2 seconds per song).
- The user interface should be intuitive and responsive.
- The system should handle large datasets efficiently without crashes.

Test objectives:

1. All field entries must work properly.
2. Pages must be activated from the identified link.
3. The entry screen, messages and responses must not be delayed.

Features to be tested

1. Verify that the entries are of the correct format.
2. No duplicate entries should be allowed.
3. All links should take the user to the correct page.

Integration Testing:

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g.

components in a software system or – one step up – software applications at the company level – interact without error.

Test Results:

All the test cases mentioned above passed successfully. No defects end counted

Acceptance Testing:

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user.

It also ensures that the system meets the functional requirements.

Test Results:

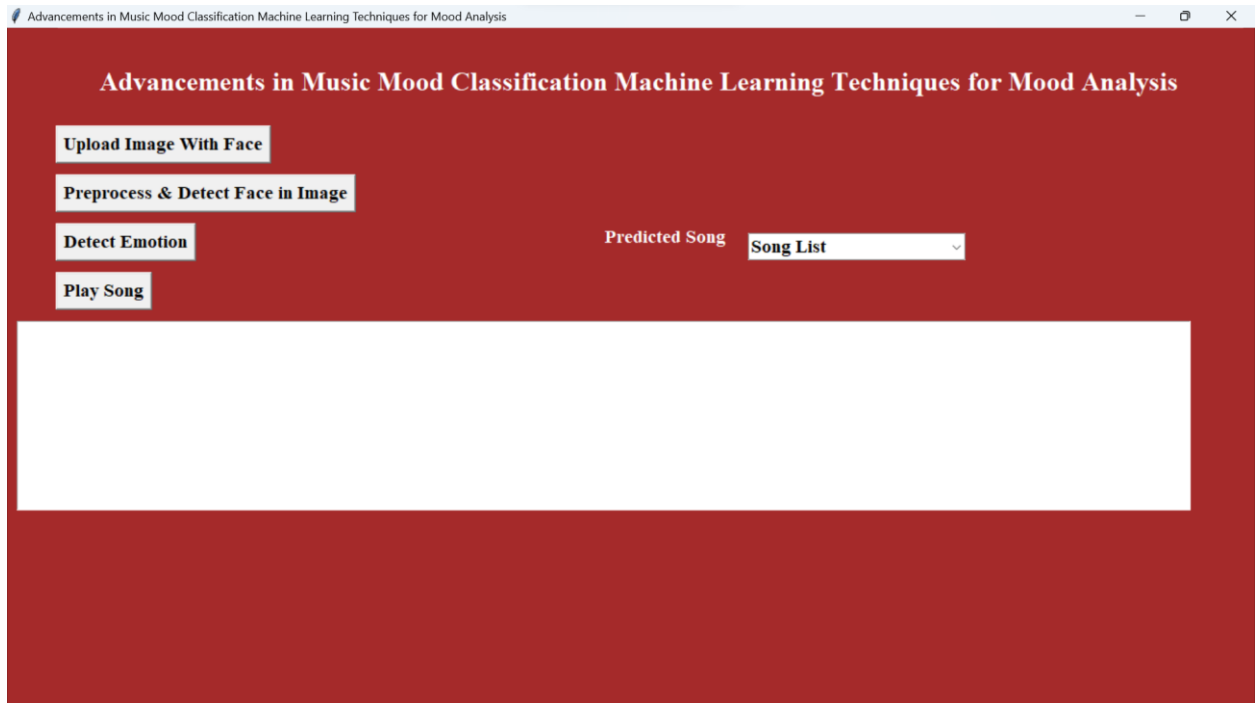
All the test cases mentioned above passed successfully. No defects encounter

7.2 SAMPLE TEST CASE:

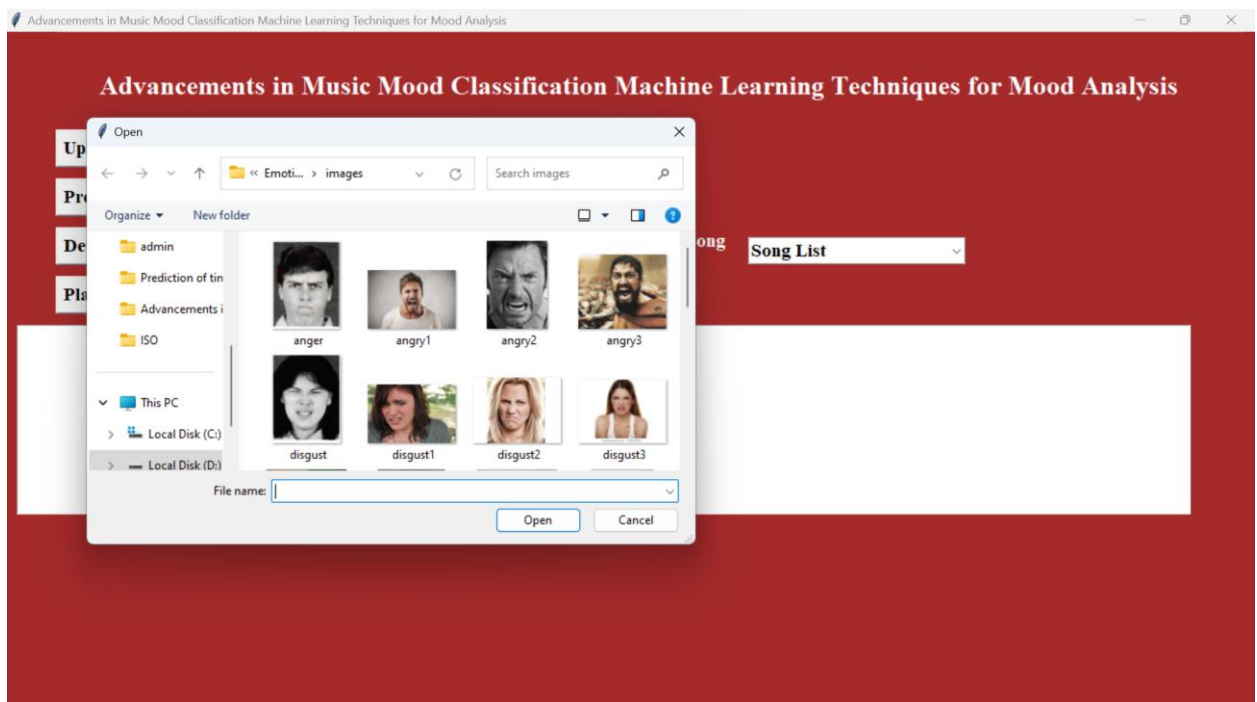
Test Case Id	Test Case Name	Test Case Desc.	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	Upload Image with Face	Test whether the Face Image is uploaded or not	If the Face Image may not uploaded	we cannot do any further operations	we can do further operations	High	High
02	Pre-process & Detect Face In Image	Test whether the Face Image is uploaded or not	Without loading the Face Image	we cannot do any further operations	we can do further operations	High	High
03	Detect Emotion	Test whether the Face Image is uploaded or not	If the Face Image may not uploaded	we cannot detect the emotion	we can detect the emotion	High	High
04	Play song	Test whether the song is played or not	If the emotion may not detected	We cannot play the song	We can play the song	High	High

CHAPTER 8: RESULT

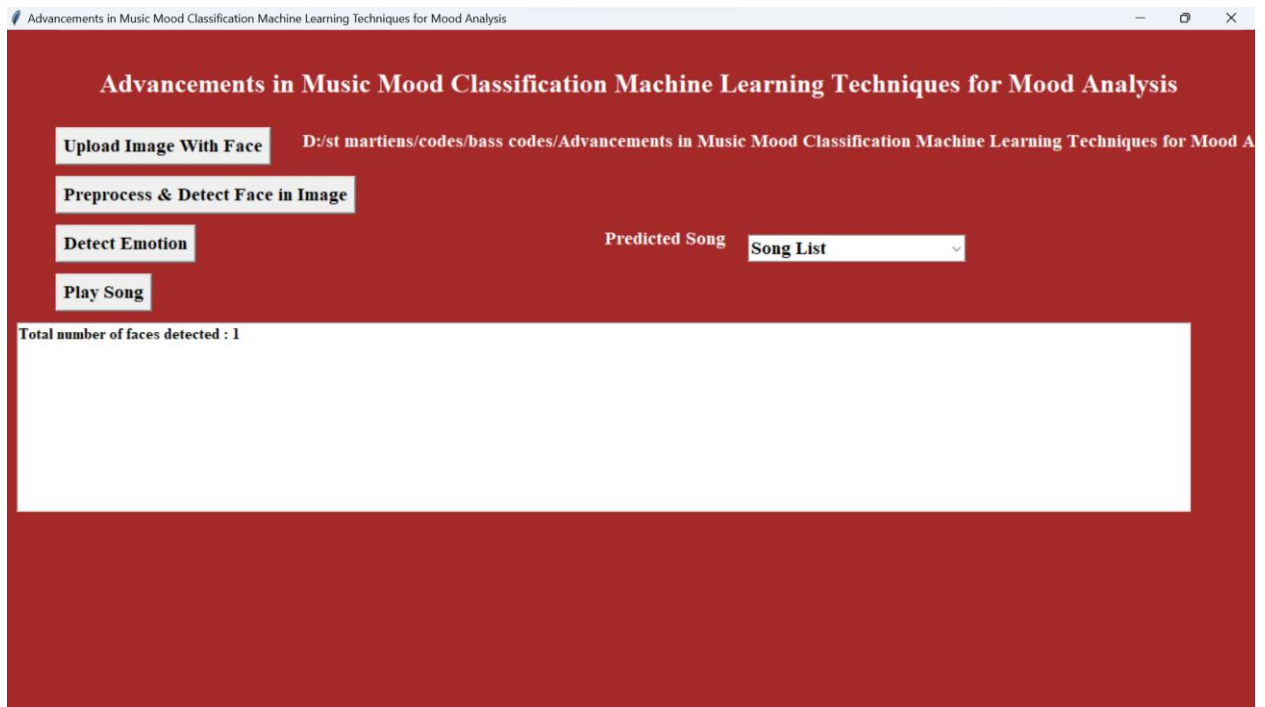
Double click on **‘run.bat’** file to get below screen



‘Upload Image with Face’



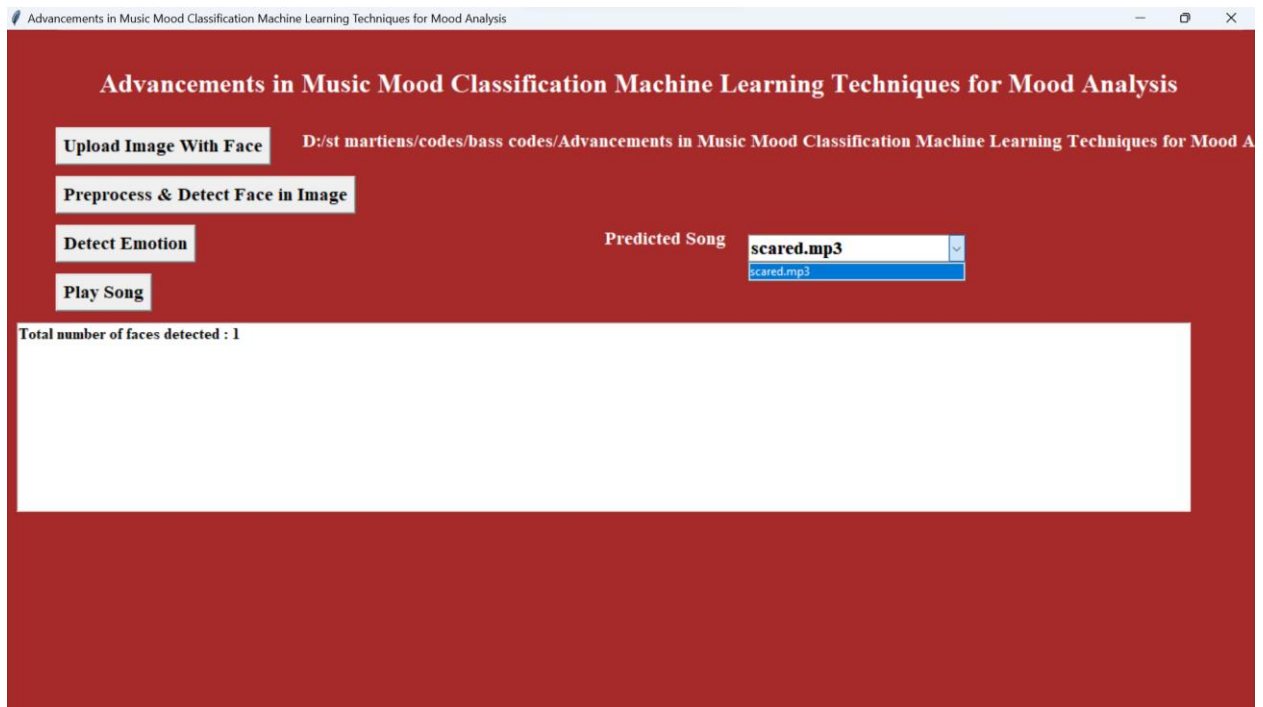
‘Pre-process & Detect Face in Image’ button to perform pre-processing and to extract face from images



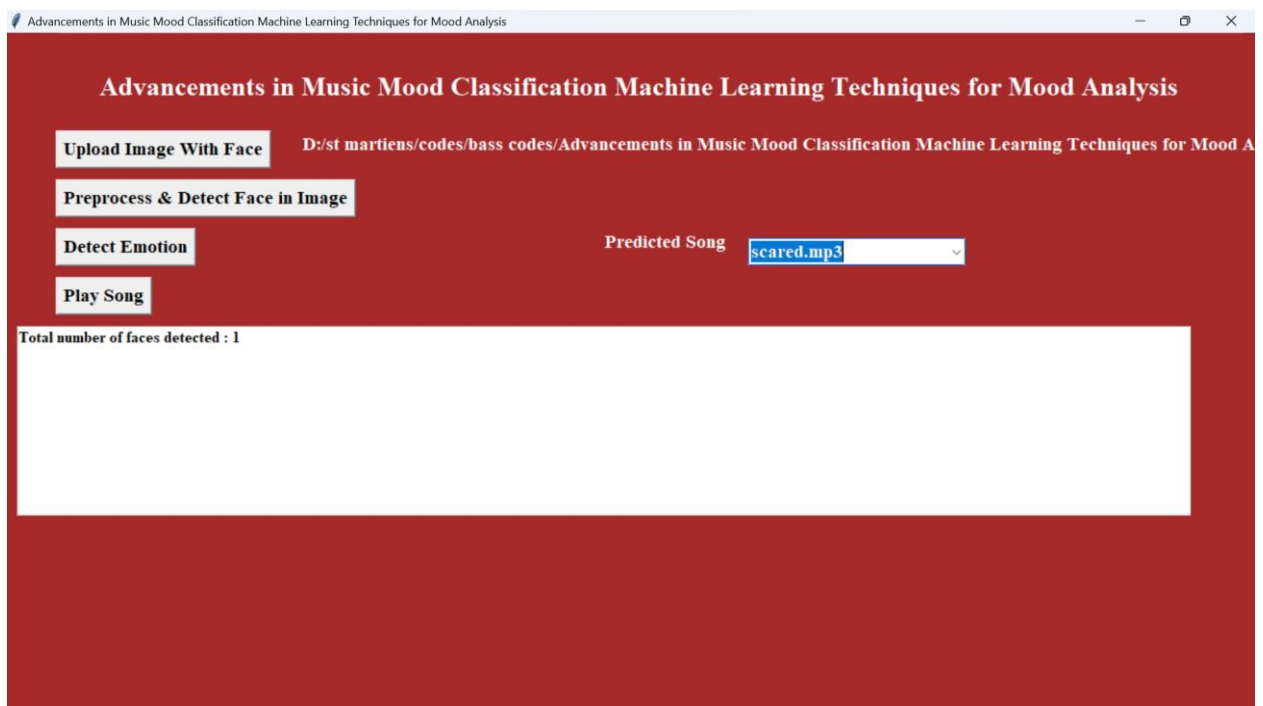
Detect Emotion button to detect emotion



click on drop down arrow link to get all disgust songs list



click on 'Play Song' button to play song



If your system has audio driver then u can hear song.

Note: no technologies can detect 100% emotion from images but this project can detect upto 90%

CHAPTER 9: CONCLUSION & FUTURE ENHANCEMENT

CONCLUSION:

The Music Mood Classification System leverages machine learning, deep learning, and NLP to classify music based on mood using audio features, lyrics sentiment, and metadata. This multimodal approach ensures high accuracy across various genres. The system undergoes extensive testing—unit, integration, performance, and security—to ensure reliability, real-time processing, and scalability for applications like music streaming and emotional AI. Explainable AI (XAI) adds transparency to mood predictions, while user feedback supports continuous improvement. Additionally, the study introduces a framework for emotion-based music recommendation using facial images to detect user mood, enhancing personalization through physiological signal analysis.

FUTURE ENHANCEMENT:

The Music Mood Classification System can be further improved with various enhancements to increase accuracy, efficiency, and user experience. Below are some potential future enhancements:

1. Deep Learning Model Optimization

- Implement advanced neural networks such as Transformers, CNN-LSTMs, and Generative AI to improve mood classification accuracy.
- Use self-supervised learning to reduce dependency on labeled datasets and enhance the model's ability to learn complex patterns.

2. Multilingual Lyrics Analysis

- Extend lyrics sentiment analysis to support multiple languages using NLP models like BERT, GPT, and mT5.
- Implement real-time translation for better mood detection in global music collections.

3. Real-Time Streaming Integration

- Enhance real-time classification capabilities to process live music streams and provide instant mood tagging.
- Enable adaptive mood detection that updates as a song progresses.

4. Emotion-Based Music Recommendations

- Develop an AI-driven recommendation system that personalizes playlists based on a user's mood, listening history, and real-time emotions.
- Integrate wearable device data (heart rate, facial expressions) to provide emotion-based music suggestions.

5. Voice and Humming-Based Search

- Allow users to search songs by humming, singing, or speaking a few lyrics using advanced speech-to-text and audio-matching algorithms.

6. Blockchain for Data Security

- Implement blockchain-based authentication and data encryption to secure user preferences, feedback, and music metadata.

7. Cross-Platform Integration

- Expand compatibility with smart devices, IoT systems, and VR/AR applications to provide immersive mood-based music experiences.

CHAPTER 10: BIBLIOGRAPHY

- [1] Shital Shankar Gujar and Ali Yawar Reha, **“Enhancing Accuracy and Performance in Music Mood Classification through Fine-Tuned Machine Learning Methods”**, and published in **2024** in the journal Communications on Applied Nonlinear Analysis.
- [2] Sheetal Shelke and Mangal Patil, **“Exploring Machine Learning Techniques for Music Emotion Classification: A Comprehensive Review”**, and was published in **February 2024**.
- [3] Anupam Garg, Vybhav Chaturvedi, Arman Beer Kaur, Vedansh Varshney, and Anshu Parashar, **“Machine Learning Model for Mapping of Music Mood and Human Emotion Based on Physiological Signals”**. It was published in **2022** in the journal Multimedia Tools and Application
- [4] Zhang, T., & Xu, M. (2020). **“Explainable music emotion classification using multi-level features and attention mechanisms”**. IEEE Access, 8, 101496–101506.
- [5] Panda, R., & Malheiro, R., & Paiva, R. P. (2018). **“Multi-modal music emotion recognition: A new dataset, methodology and comparison”**. In Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR), 571–577.
- [6] D. Ayata, Y. Yaslan, and M. Kamasak, **“Emotion recognition via galvanic skin response: Comparison of machine learning algorithms and feature extraction methods,”** IU J. of Elect. & Elect. Eng., vol. 17, no. 1, pp. 3129–3136, 2017.
- [7] F. Isinkaye, Y. Folajimi, and B. Ojokoh, **“Recommendation systems: Principles, method and evaluation,”** Egypt. Inf. J., vol. 16, no. 3, pp. 261–273, 2015.
- [8] R. L. Rosa, D. Z. Rodriguez, and G. Bressan, **“Music recommendation system based on user’s sentiments extracted from social networks,”** IEEE Trans. Consum. Electron, vol. 61, no. 3, pp. 359–367, Aug 2015.
- [9] S. Jhaharia, S. Pal, and S. Verma, **“Wearable computing and its application,”** Int. J. Comp. Sci. and Inf. Tech., vol. 5, no. 4, pp. 5700– 5704, 2014.
- [10] K. Popat and P. Sharma, **“Wearable computer applications: A feature perspective,”** Int. J. Eng. and Innov. Tech., vol. 3, no. 1, 2013.

- [11] S. Nirjon, R. F. Dickerson, Q. Li, P. Asare, J. A. Stankovic, D. Hong, B. Zhang, X. Jiang, G. Shen, and F. Zhao, “**Musical heart: A hearty way of listening to music,**” in Proc. of ACM Conf. on Emb. Newt. Sens. Sys. ACM, 2012, pp. 43–56.
- [12] K. Yoon, J. Lee, and M. U. Kim, “**Music recommendation system using emotion triggering low-level features,**” IEEE Trans. Consum. Electron, vol. 58, no. 2, pp. 612–618, May 2012.
- [13] P. Melville and V. Sindhwani, “**Recommender systems,**” in Encyc. of mach. learn. Springer, 2011, pp. 829–838.
- [14] H. Liu, J. Hu, and M. Rauterberg, “**Music playlist recommendation based on user heartbeat and music preference,**” in Int. Conf. on Comp. Tech. and Dev., vol. 1. IEEE, 2009, pp. 545–549.
- [15] N. Sebe, I. Cohen, T. S. Huang et al., “**Multimodal emotion recognition,**” Handbook of Pattern Recognition and Computer Vision, vol. 4, pp. 387–419, 2005.
- [16] A. Nakasone, H. Prendinger, and M. Ishizuka, “**Emotion recognition from electromyography and skin conductance,**” in Proc. of Int. Work. on Bio signal Interp., 2005, pp. 219–222.
- [17] J. B. Schafer, “**Dynamic lens: A dynamic user-interface for a meta recommendation system,**” in Work. on Bey. Pers., 2005, pp. 72–76.
- [18] R. W. Picard, E. Vyzas, and J. Healey, “**Toward machine emotional intelligence: Analysis of affective physiological state,**” IEEE Trans. Pattern Anal. Mach. Intel., vol. 23, no. 10, pp. 1175–1191, 2001.

CHAPTER 11: YUKTHI INNOVATION CERITICATE